The goal of Starter Project 3 was to practice professional software engineering workflows using GitHub branches, pull requests, peer code review, and static analysis tools. The assignment required submitting code through a pull request, reviewing a peer's implementation, receiving feedback on my own code, and then improving readability, correctness, and maintainability based on that feedback. Instead of coding to just get the right output, this assignment focused more on writing code that can survive collaboration, scrutiny, and future modification.

I developed my solution by copying my existing implementation from Starter Project 2 into the Starter Project 3 Codio and then following the required branching, committing, and pushing steps for GitHub outlined in the assignment. Working in a separate branch ensured that my changes were isolated from the main branch while I prepared the submission. After making sure that the code ran correctly in this environment, I submitted it through a pull request. Instead of constantly submitting my code to Codio where if you passed the set test cases you were okay, for this assignment now my code really had to be readable and understandable to another person. I was nervous when I went to request my as after putting my code into GitHub I felt as though it made all of my weaknesses in formatting, and structure within my code more visible as the code was now written for external review rather than for an auto check Codio review.

During the code review process, the most significant feedback I received concerned my dictionary preprocessing logic, specifically how prefixes were generated and stored. While my implementation correctly identified prefixes and full words for the provided test cases, my reviewer pointed out that my design didn't handle empty strings as a valid prefix. Although this did not cause failures in the current tests, the concern was valid as excluding the empty prefix introduces a tighter coupling between the DFS initialization and dictionary structure, which in turn could result in making the code more fragile to future changes along the line. This feedback was something I hadn't considered throughout the coding process and it helped me better understand and see first hand how code should not only work for known inputs but should also be resilient to reasonable variations in usage.

In my review of my partner's code, I focused on design clarity, if validation for the grid and dictionary was properly met, and long-term maintainability. I offered several targeted suggestions to improve the quality and clarity of their Boggle code, such as adding explicit checks for empty rows within the grid, enforcing consistent column counts across all rows, checking for if special characters or numbers were entered in the dictionary and or grid, and avoiding the inclusion of full words in their prefix set. I also recommended improving naming clarity for directional variables and replacing the "magic numbers" with named constants to make the code easier to modify in the future.

Based on my peer review feedback, I renamed methods to follow Python's snake_case convention as that's how my variables were named originally to make the code more consistent

and easier to read. In set_dictionary, I initialized self.dict_hash with an empty string key to prevent the DFS from stopping early on empty prefixes. I also simplified validation checks by replacing if dictionary == [] with if not dictionary. These changes preserved the original algorithmic logic but improved readability, prevented the subtle DFS errors, and aligned the code with professional Python standards.

In addition to peer feedback, static analysis exposed a large number of formatting and style issues. The linter flagged inconsistent indentation, improper spacing around operators and commas, incorrect comment formatting, and excessive or missing blank lines. None of these issues affected runtime behavior, but collectively they reduced readability and violated Python's PEP 8 conventions. Seeing the volume of warnings made it clear how quickly small style inconsistencies can accumulate and how difficult they become to manage without automated tooling as it was incredibly time consuming to fix all the errors and for me was the hardest and most grueling part of Starter Project 3. Instead of using tab I had to go back and change all my indentation to four space increments, I reformatted comments to follow conventional style, and cleaned up all of the unnecessary blank lines I had. While these changes did not alter the algorithm itself and seemed useless to me in the moment as I went through them all, they substantially improved the readability and professionalism of my code

After implementing these improvements, I went to rerun the unittests to ensure no regressions were introduced and unfortunately had to go through all of the same formatting issues I had with my Boggle solver again with my unittests before I could run them as there was now a ton of errors since the formatting of the tabs messed things up. After formatting everything, almost all of the tests passed successfully. As I went to run them I started to have tests fail that didn't fail in the pass and I had to actually go back to my Boggle solver and fix some of the indentation that I made because after formatting everything, some lines got out of place and out of loops. Despite the tedious nature of it all it made me realize the importance of regression testing as a safeguard when making structural or stylistic changes, especially in collaborative environments where unintended side effects can easily slip in.

Reflecting on the entire process, this project demonstrated how software engineering extends well beyond producing correct output. Peer review forced me to justify design decisions I initially took for granted, and revealed weaknesses in my code that were not immediately obvious during solo development. Static analysis provided an objective standard that eliminated ambiguity about style and best practices. Most importantly, the iterative loop of feedback, refactoring, and testing mirrored real-world development workflows, where code quality, clarity, and maintainability matter just as much as correctness. This experience strengthened my ability to write code that works, but is also and most importantly robust, readable, and able to be understood by others.